

File System

Estructura para representar entradas de directorio

```
struct Ext2FSDirEntry {
    unsigned int inode ;
    unsigned short record_length ;
    unsigned char name_length;
    unsigned char file_type ;
    char name [ ] ;
};
```

Estructura para representar inodos

```
struct Ext2FSInode {
    unsigned short mode; // info sobre el tipo de archivo y los permisos
    unsigned short uid ; // id de usuario
    unsigned int size ; // en bytes
    unsigned int atime ;
    unsigned int ctime ;
    unsigned int mtime; // fecha ultima modificacion
    unsigned int dtime ;
    unsigned short gid ; // id de grupo
    unsigned short links_count ; // cantidad de enlaces al archivo
    unsigned int blocks ;
    unsigned int flags ;
    unsigned int os_dependant_1;
    unsigned int block [ 15 ] ;
    unsigned int generation ;
    unsigned int file_acl ;
    unsigned int directory_acl ;
    unsigned int faddr ;
    unsigned int os_dependant_2 [ 3 ] ;
};
```

Funciones basicas

- Dado el id de un usuario , devuelve el nombre del usuario.

```
char * nombre_propietario(unsigned short id_usuario)
```

- Dado un path , devuelve su inodo

```
struct Ext2FSInode * Ext2FS :: inode_for_path(const char * path)
```

- Dada una direccion de bloque y un buffer , carga el bloque indicado en el buffer

```
void Ext2FS :: read_block(unsigned int block_address , unsigned char * buffer )
```

- Dados un inodo y un numero de bloque , recorre el inodo buscando la direccion del bloque de datos indicado

```
unsigned int Ext2FS :: get_block_address(struct Ext2FSInode * inode , unsigned int block_number)
```

- Dado un numero de inodo , busca el inodo en el grupo y lo devuelve

```
struct Ext2FSInode * Ext2FS :: load_inode(unsigned int inode_number)
```

Funciones

```
find_file_less_size(char* dir, int min_byte, char* arch_name){
```

```
    ffls_rec(Ext2FSInode* inode, int min_bytes, char* arch_name){
```

```
        char* buf = malloc(2*BLOCK_SIZE);
```

```
        int cant_dentry = 0;
```

```
        int block_number = 0;
```

```
        int recorrido = 0;
```

```
        int block_addr = get_block_address(inode, block_number);
```

```
        int offset = 0;
```

```
        read_block(block_addr, buf);
```

```
        read_block(block_addr, buf + BLOCK_SIZE);
```

```
        while(recorrido < inode.size){
```

```
            // Ajuste de buffer
```

```
            Ext2FSDirEntry dentry* = (Ext2FSDirEntry*) buf + offset;
```

```
            recorrido += dentry->size
```

```

        if (!strcmp(dentry->name, name, dentry->name_lenght)){
            Ext2FSInode* dir_inode = load_inode(dentry->inode);
            if (dir_inode->size <= min_bytes) {
                printf(dentry->name);
            }
        }
        if (dentry->file_type == "d" && cant_dentry >= 2){
            Ext2FSInode* inode = load_inode(dentry->inode);

            ffls_rec(inode, min_bytes, arch_name);
        }

        if (dentry->file_type == "d" && cant_dentry < 2){
            cant_dentry ++;
        }

        // Actualizo punteros, contadores, etc

        offset += dentry->size;
        if (offset >= BLOCK_SIZE){
            offset -= BLOCK_SIZE;
            read_block(get_block_addr(inode, block_number++), buf);

            if (inode->size - recorrido > BLOCK_SIZE) {
                read_block(get_block_addr(inode, block_number+
1), buf);
            }
        }
    }
}

Ext2FSInode* inode = inode_for_path(dir);

ffls_rec(inode, min_bytes, arch_name);
}

```